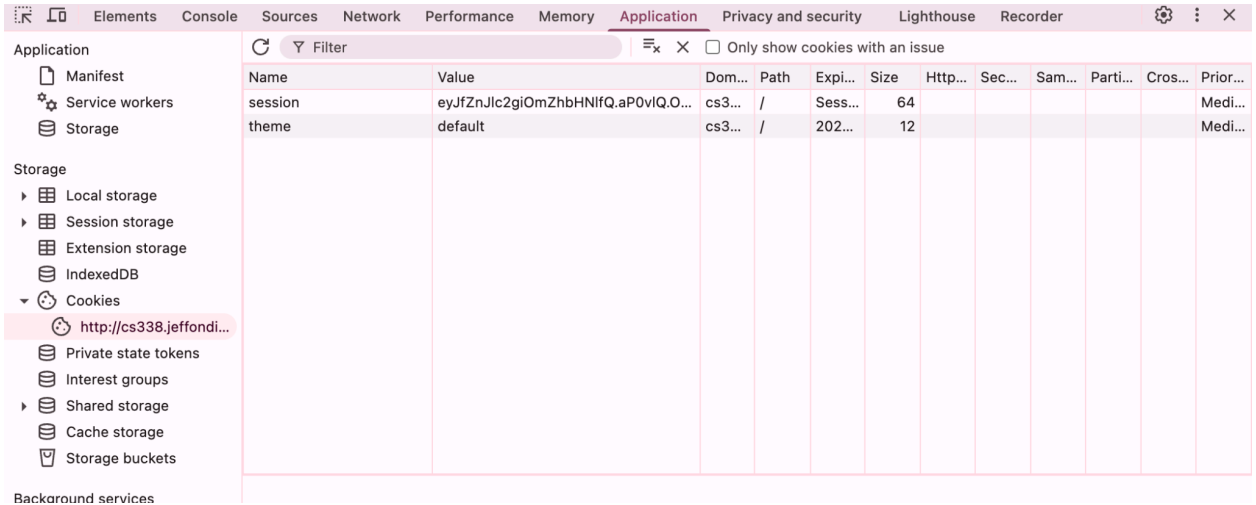


Cookies and Cross-Site Scripting (XSS)

Beshir Said

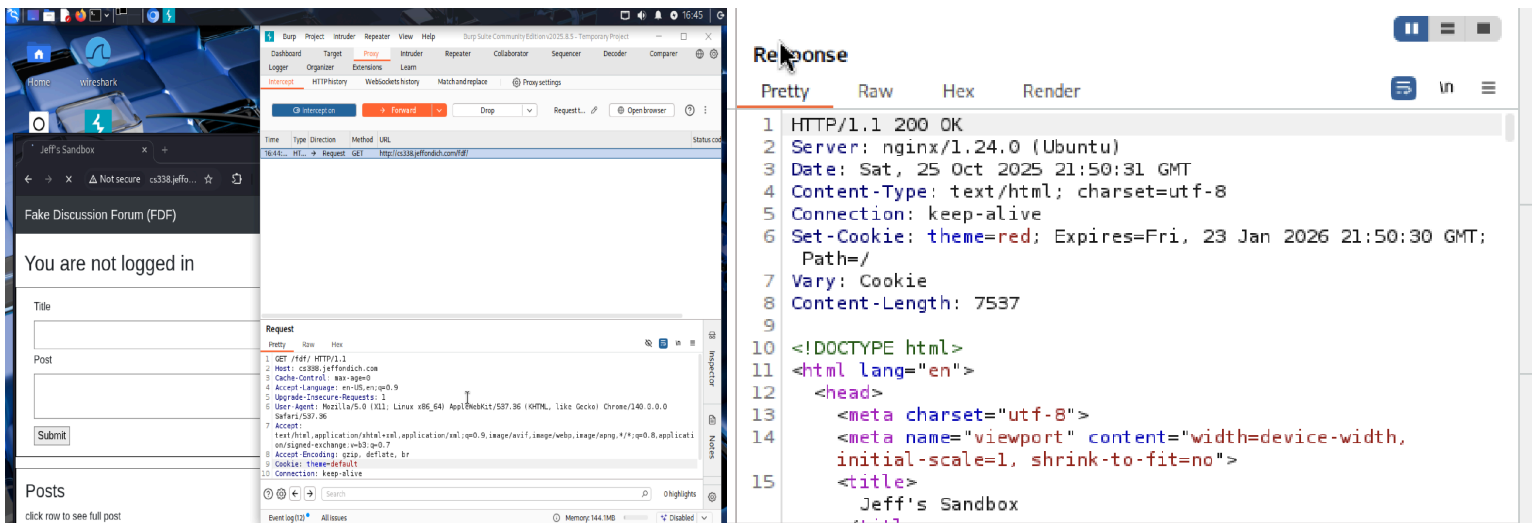
Part 1: Cookies

- a) For the domain, there's a session cookie with a value in a base64 string and a theme cookie with a default value.



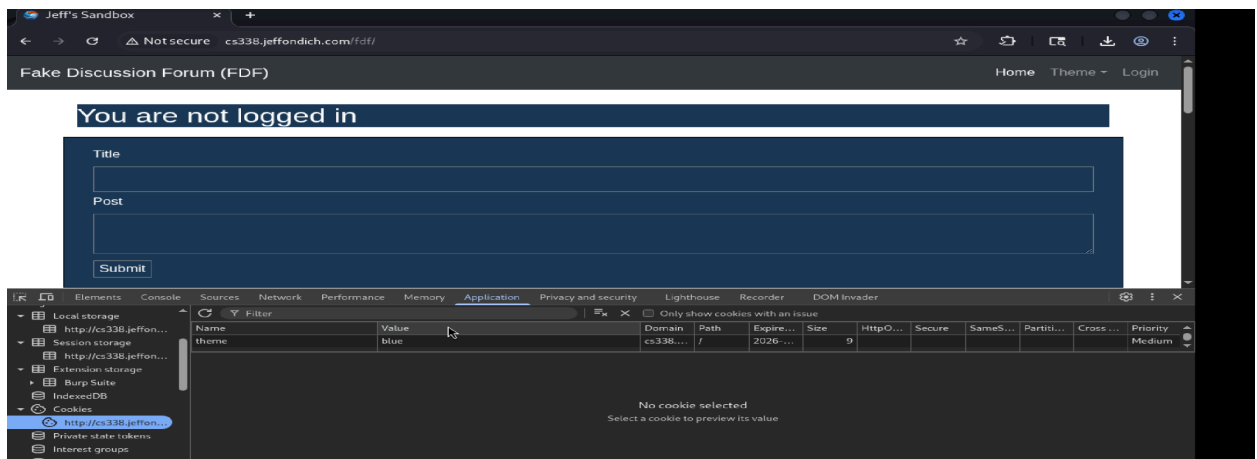
Name	Value	Dom...	Path	Expi...	Size	Http...	Sec...	Sam...	Parti...	Cros...	Prior...
session	eyJfZnJlc2giOmZhbnRlbnQ.aP0vIQ.O...	cs3...	/	Sess...	64						Medi...
theme	default	cs3...	/	202...	12						Medi...

- b) The value for the theme cookie changed into the name of the theme(i.e when I switched the theme to red, the value became red; etc.) The size of the cookie also changed. For the default theme, the size was 12 bytes. For the red theme, the size was 8 bytes. For the blue theme, the size was 9 bytes.
- c) Using Burp Suite, I captured the HTTP request and response between my browser and the FDF server. The request contained a Cookie header showing that the browser was sending its cookie values to the server. This matched the cookie information from the browser's Inspector. After I changed the theme color, the server's response included a new Set-Cookie header. This instructed the browser to update the cookie value to reflect the new theme. The FDF server uses the Set-Cookie headers to store and update user preferences, in this case themes.



The screenshot shows Burp Suite's interface with a captured HTTP request and response. The request is from http://cs338.jeffondl.com/fdf/. The response is from 192.168.1.100 and includes a Set-Cookie header: Set-Cookie: theme=red; Expires=Fri, 23 Jan 2026 21:50:30 GMT; Path=/. The response body shows an HTML page titled 'Jeff's Sandbox'.

- d) After closing and reopening the Burp Suite browser, the FDF automatically loaded the red theme I selected last. This shows that the browser stores the Set-Cookie and reapplies the preference on future visits. This is explained by the cookies additional information Expires which is set to 2026, making this a persistent cookie.
- e) The current theme is transmitted with every HTTP request. The Cookie sends the user's current theme value to the server and thus the theme value is transmitted through the Cookie header in each HTTP request. So whenever the page is loaded, the browser automatically includes the Cookie and theme with value in the request headers letting the server know to render the correct theme
- f) When you change the theme, the browser sends an HTTP GET request and the server responds with a Set-Cookie header that updates the theme value. Thus, the change is transmitted in the theme value. The browser stores that cookie and uses it in future requests.
- g) You can manually change the theme by editing the cookie value to your preferred color and refreshing the page updates the theme.
- h) In Burp Suite, you intercept the request and manually change the cookie value to your preferred theme and forward it to the server.
- i) The OS stores the cookies inside the cookie data files in the browsers. For Firefox, the cookies are stored locally and you can find it in the Store Inspector.



Part 2: Cross-Site Scripting(XSS)

- a) There are 3 types of XSS attacks. A Stored or Persistent XSS happens when an attacker injects malicious JS code into a target (forum or database) and every viewer executes the malicious code. A Reflected XSS is where the attacker's input is included in a url or form. The victim would need to click on the link for the attack to execute. Phishing emails are a good example of reflected XSS. A DOM-based XSS is a client side JS attack where the victim reads the attackers data and writes it into a Document Object Model

(DOM) maliciously. This is stored in the browser code not on the server.

Link: <https://www.acunetix.com/websitesecurity/xss/>

- b) Prof. Moriarty submits a post containing a JS/CSS script. Then the FDF server stores and serves the post onto the forum. Then the victim using the browser executes the script. Then the script actions appear on the victim's browser. This is a Stored XSS attack since the malicious JS appears in the stored post body and executes to anyone who views the post.
- c) The attacker could inject a script that reads and steals authentication tokens or session tokens. This is extremely problematic because the attacker can then access private data, impersonate the victim and change account settings. This is worse than the theme color change and alert since it doesn't have the same consequences as data theft, it's just a UI bother.
- d) Another attack could be on an active session to change the server state. The attacker could inject JS issues that perform an action while the victim is logged in, such as changing password or transferring funds and since the requests are from the victim's browser, they would automatically carry the victim's cookies.
- e) When browsing, avoid storing sensitive tokens in local storage since JS can easily access them through XSS. When working on any type of application, make sure the user-provided content is sanitized to remove malicious code. Also use frameworks that escape user data to prevent attackers from manipulating with raw input.